AI-Powered Test-Case Generation App using IBM Bob

Mar-2026

Jeya Gandhi Rajan M
Senior Solution Architect
CE Ecosystem

Pratham S Shetty
Platform Engineer
CE Ecosystem

Surya Prakash
Platform Engineer
CE Ecosystem

Use case

Problem

- IBM Flash Storage teams face an extremely high testing workload
- NVMe specification alone spans **784 pages**, making manual interpretation slow and error-prone
- Creating comprehensive **test cases** manually takes **3–4 months**
- Every specification update forces teams to **repeat** large portions of the work

Pain Points

- Inconsistent **test coverage** across different teams
- Up to **60% of testing time** spent purely on test-case creation
- Knowledge locked in individual engineers (**knowledge silos**)
- High maintenance effort for frequent specification revisions
- Repetitive manual steps converting written test cases into executable scripts

Solution

- Using IBM Bob, we built an **AI-Powered Test Case Generation App**
- The app transforms the months-long process of interpreting massive technical specs (e.g., the 784-page NVMe document) into an automated AI-driven workflow
- Powered by **watsonx.ai**, the solution delivers faster, smarter, and more reliable test creation
- This enables teams to focus on innovation instead of manual documentation work
- Outcomes include **accelerated release cycles**, improved product **quality**, and **reduced QA costs**
- Core Capabilities
  - Intelligent **Document Processing** :Automatically extracts and understands PDF specifications (NVMe, PCIe, etc.)
  - AI-Driven **Test-Case Generation**: Generates structured, consistent, and comprehensive test cases
  - Enterprise-Grade **Security**: Designed to operate securely within enterprise environments

Figure 146: Abort – Command Dword 10

Bits	Description
31:16	**Command Identifier (CID):** This field specifies the command identifier of the command to be aborted, that was specified in the CDW0.CID field within the command itself.
15:00	**Submission Queue Identifier (SQID):** This field specifies the identifier of the Submission Queue that the command to be aborted is associated with.

5.2.1.1 Immediate Abort requirements

If the controller performs an immediate abort, then the controller shall ensure that, other than the posting of the completion queue entry for the command to abort, there are no effects of that command (e.g., no host memory access, command processing does not modify: NVM media, NVM Set state, Endurance Group state, namespace state, controller state, domain state, or NVM subsystem state) subsequent to the posting of the CQE for the command that requested the abort. If the controller is not able to ensure there are no effects of the command to abort subsequent to the posting of the CQE for the command that requested the abort, then the controller shall not perform an immediate abort on that command.

For example, if a command:

- requests a data transfer;
- the data transfer has been initiated and not completed; and
- the CQE for that command was not posted prior to the posting of the CQE for the command that requested the abort,

then the controller is prohibited from performing an immediate abort.

Performing an immediate abort effects the information returned in the CQE for the command that requested the abort as described in section 5.2.1.2 for an Abort command and in section 7.1.1 for a Cancel command.

5.2.1.2 Command Completion

Upon completion of the Abort command, the controller posts a completion queue entry to the Admin Completion Queue indicating the status for the Abort command. Dword 0 of the completion queue entry indicates information about the command to abort as shown in Figure 147.

Figure 147: Abort Command – Completion Queue Entry Dword 0

Bits	Description
31:01	Reserved
00	**Immediate Abort Not Performed (IANP):** Indicates whether or not an immediate abort was performed. If this bit is set to '1', then an immediate abort was not performed (i.e., the command to abort was not aborted prior to posting the CQE for the Abort command) for any reason (e.g., the immediate abort requirements were not met, the requested command to abort was not found). If this bit is cleared to '0', then an immediate abort was performed (i.e., the command to abort was aborted prior to posting the CQE for the Abort command).

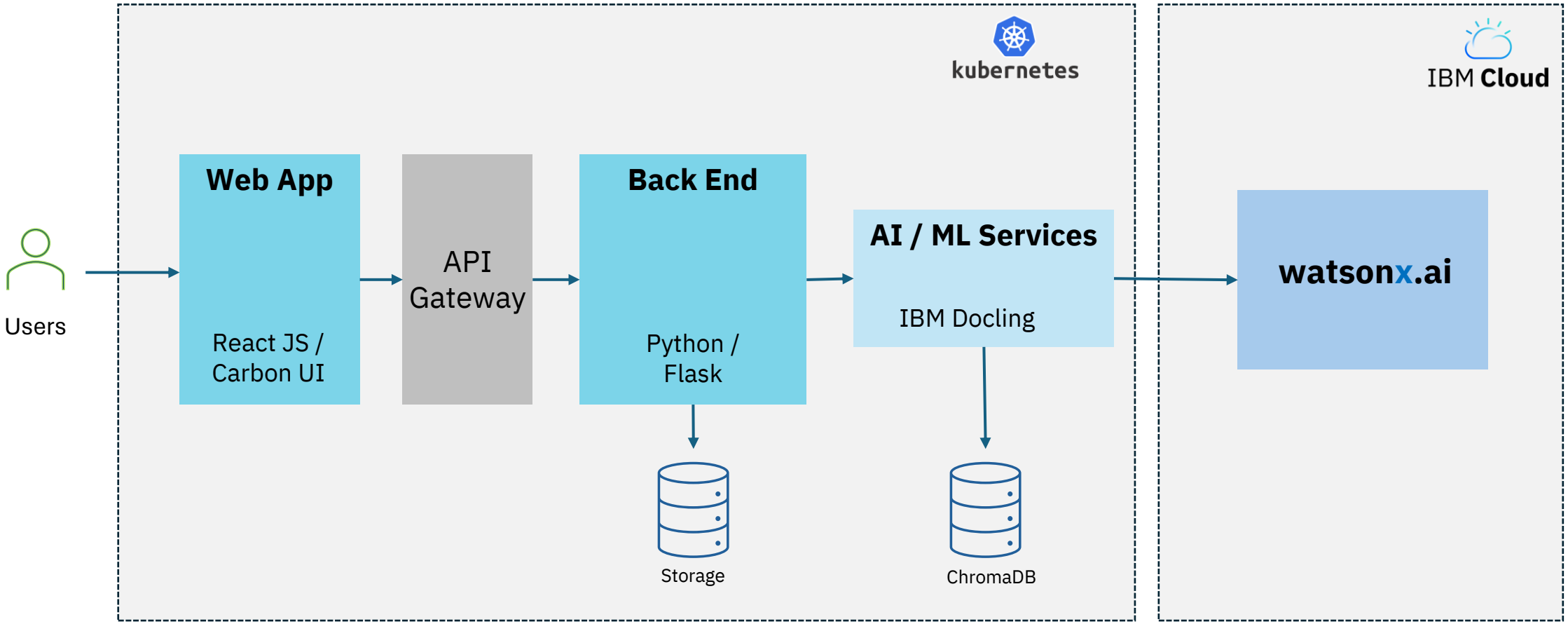
If the Immediate Abort Not Performed bit in Dword 0 (refer to Figure 147) is set to '1' in the completion queue entry for the Abort command and a deferred abort is performed, then the controller shall set the status code for the command to abort to Command Abort Requested. The host should examine the status in the completion queue entry of the command to abort to determine if the command was aborted.

Command specific status code values associated with the Abort command are defined in Figure 148.

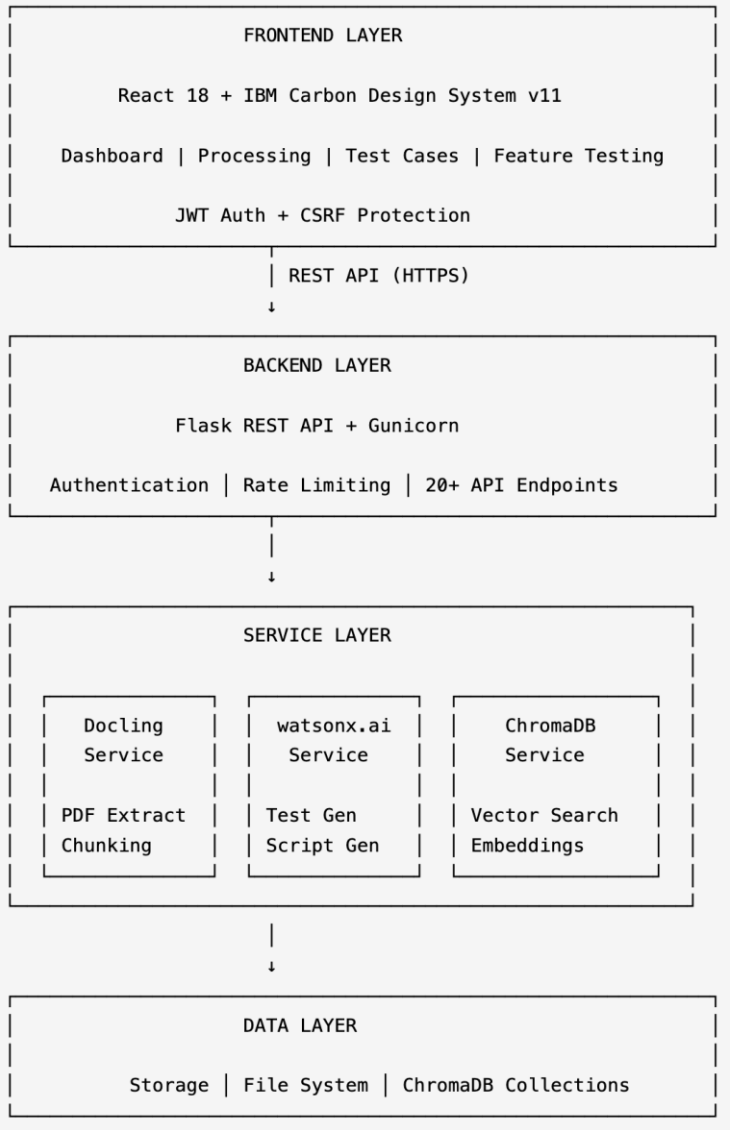
Figure 148: Abort – Command Specific Status Values

Value	Definition
3h	**Abort Command Limit Exceeded:** The number of concurrently outstanding Abort commands has exceeded the limit indicated in the Identify Controller data structure.

Architecture



Tech Stack



Component	Technology
Frontend	React 18 + Carbon Design System
Backend	Flask + Python 3.9+
AI Platform	IBM watsonx.ai
Document Processing	IBM Docling
Vector Database	ChromaDB
Authentication	JWT + CSRF Protection
Rate Limiting	Flask-Limiter
Deployment	IBM Cloud

Outcome and Best Practice

Outcomes

Development Metrics

- 10,000+ lines of production code
- 1,200+ lines of automated tests
- 8,000+ lines of documentation
- 75% time reduction with IBM Bob

Performance Metrics

- 30 seconds for 10 test cases (vs. 2 hours manually)
- 15 seconds per test script (vs. 30 minutes manually)
- 90%+ time savings in test case creation

Quality Metrics

- 95% of generated test cases meet quality standards
- 99.5% uptime during testing period
- 100% documentation coverage

Best Practices

Orchestrator Mode for Project Management

- Broke down project into 95+ manageable tasks
- Coordinated phase transitions (Planning → Implementation → Testing → Documentation)
- Delegated to specialized modes (Plan, Code, Advanced)

Prompt Engineering Excellence

- Structured templates with clear sections
- Validation rules for consistency
- Edge case generation instructions

Security-First Approach

- JWT authentication with refresh tokens
- CSRF protection for auth endpoints
- Rate limiting per endpoint category

Comprehensive Testing Strategy

- Backend 940+ lines (Pytest)
- Frontend 279+ lines (Vitest)
- Integration tests for critical paths
- Edge case coverage

Business Value

QA & Test Engineers

Time Savings
90%+ reduction in test case creation time

Quality Improvement
Consistent, comprehensive test coverage

Focus Shift
More time for actual testing vs. documentation

Example
NVMe testing reduced from 3-4 months to 1-2 weeks

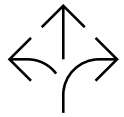
Development Teams

Faster Releases
Accelerated testing cycles

Better Coverage
AI identifies edge cases humans might miss

Multi-Language Support
Scripts in Python, Java, JavaScript, TypeScript

Example
Generate test suites for new features in minutes



Product Managers

Cost Reduction
75% less time = 75% cost savings

Faster Time-to-Market
Weeks saved in testing phase

Risk Mitigation
Comprehensive test coverage reduces bugs

Example
Launch products 2-3 months earlier

Enterprise Organizations

Scalability
Handle multiple specification documents simultaneously

Standardization
Consistent test quality across teams

Knowledge Preservation:
Test cases stored and searchable

Example
IBM Flash Storage teams can scale testing capacity

Motion 3

https://github.ibm.com/apac-ce/bob-a-thon-TestCaseGenerator

Thank you